

Hard Binary Linear Systems: Where the v6 Pipeline Stands, What Failed, and What to Reinforce

neuroMCTS project · Research meeting · July 2026

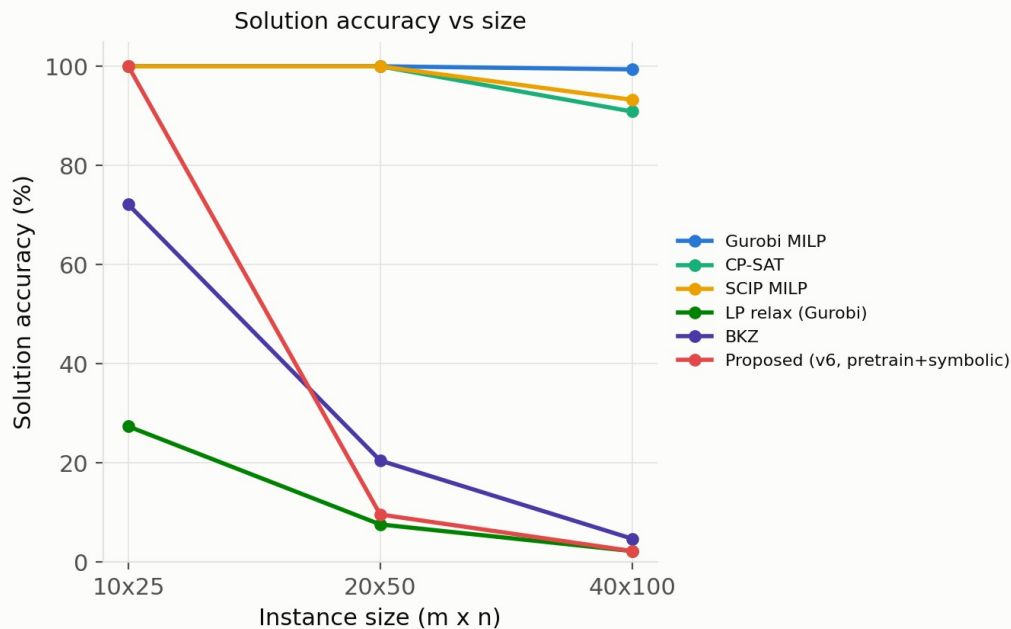
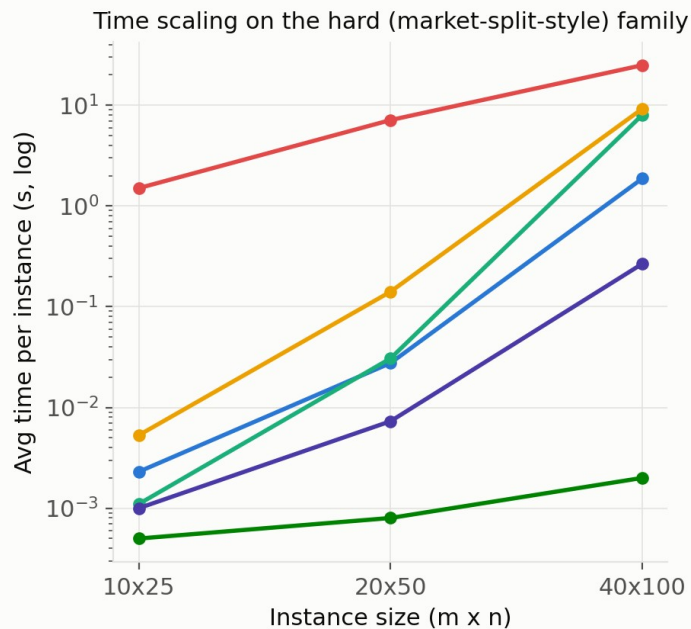
The old test family was easy: exact solvers closed it in milliseconds

Old DNS family: b leaked the planted density; 61% of instances closed before any neural inference

New market-split-style family: half-density planting hides b ; infeasible = ± 1 noise, CP-SAT-certified

At 20×50 the symbolic front end closes almost nothing — learned search must carry the load

Exact solvers' time explodes with size — and they start failing at 40×100



800–7,300× time growth across the sweep; CP-SAT leaves 9% of feasible 40×100 unsolved (60 s limit)

Our pipeline saturates 10×25, then falls behind: the honest scoreboard

10×25: feas 99.3% / sol 100% at 1.50 s — dominates every inexact baseline

20×50: feas 82.5% / sol 9.6% at 7.1 s — behind BKZ (20.5%) on solutions

40×100: feas 80.2% / sol 2.3% at 25.0 s — only the kernel pump still contributes

Zero false negatives at every size; pump cost stays in milliseconds at every size

Negative result 1: self-play RL made test search worse, and we can show why

20×50 solution accuracy: pretrained 9.6% → after RL 5.9% (same budgets)

Telemetry: selection entropy collapsed 3.8 → 0.65 while full-difficulty success stayed low

Reward starvation was repaired (dense reward + reverse curriculum), training became healthy — but gains did not transfer to full difficulty

Negative result 2: feasibility cannot be classified from static features

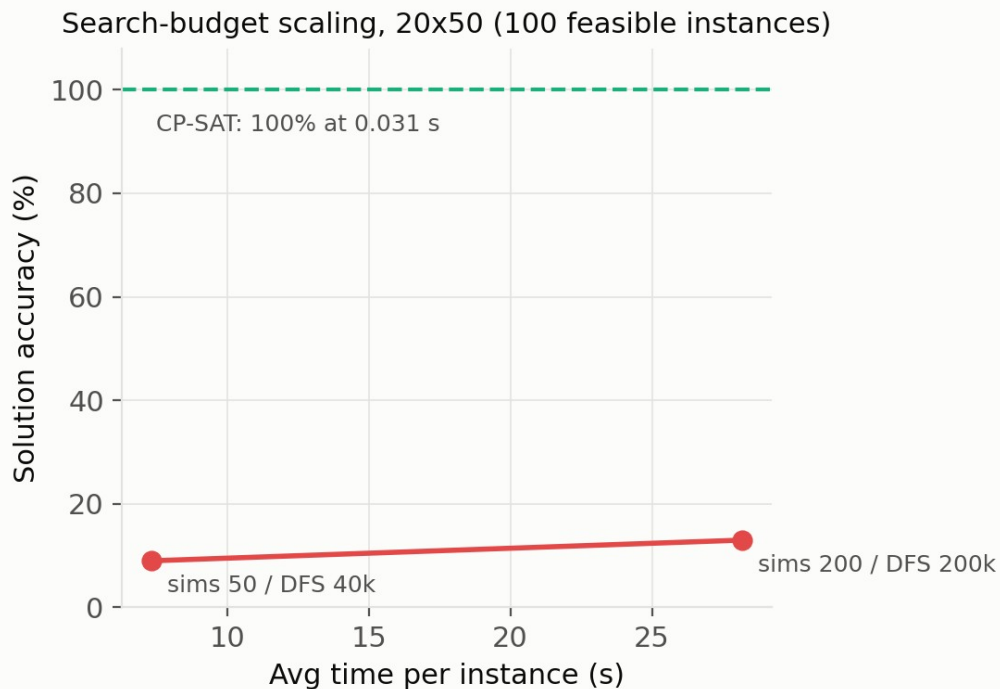
Fixed infeasible pool $\rightarrow$ head learned instance identity (novelty shortcut), 0 test rejections

Fresh both classes + head-only fine-tune $\rightarrow$ converges to $p \approx 0.5$ everywhere (focal loss at its 0.173 fixed point)

Consistent with hardness: a cheap separator for ± 1 -perturbed systems would contradict NP-hardness

Conclusion: feasibility detection must be proof-based (search-derived), not classified

4× search budget buys 4 points: the constraint is search technology, not compute



Key comparison

Same 100 instances: 9 → 13
solved for 7.3 s → 28.2 s

CP-SAT: 100% at 0.031 s on
the same instances

Its edge is conflict learning and
propagation — mechanisms
our DFS lacks

Reinforcement roadmap: lattice enumeration first, conflict learning second

1 Lattice-reformulated search: branch in the LLL-reduced kernel basis (Aardal 2000; Wassermann 2025) with learned ordering — the project's core novelty, highest headroom

2 Conflict learning in DFS: no-good recording + restarts — bounded engineering, largest cost/benefit

3 Learning inside exact search: imitate strong branching (learn2branch); drop open-loop self-play

Not reinforcing: more self-play RL, raw budget, ordering-REINFORCE at small budgets

The research question stays alive precisely where exact solvers begin to fail

Hard family validated: heuristics collapse, exact-solver time explodes, first exact failures at 40×100

Honest standing: we win at 10×25 , lose beyond — with two diagnosed negative results guiding redesign

Next cycle: no-good DFS (days), then lattice-basis enumeration with learned guidance (weeks)

References

Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., & Smeltink, J. W. (2000). Market split and basis reduction. *INFORMS Journal on Computing*, 12(3), 192–202.

Cornuéjols, G., & Dawande, M. (1998). A class of hard small 0–1 programs. *IPCO 1998*, 284–293.

Fischetti, M., Glover, F., & Lodi, A. (2005). The feasibility pump. *Mathematical Programming*, 104(1), 91–104.

Gasse, M., Chételat, D., Ferroni, N., Charlin, L., & Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. *NeurIPS* 32.

Wassermann, A. (2025). Solving the market split problem with lattice enumeration. *arXiv:2508.08702*.